Processor Verification
Final Project: Greatest Common Divisor

Lorenzo Pattaro Zonta

June 23, 2026

1 Design Under Test - Description

The GCD module computes the Greatest Common Divisor of two unsigned integers utilizing the subtractive Euclidean algorithm. It processes one pair of inputs at a time and interfaces with other components via two independent ready and valid handshake channels.

1.1 Mathematical Definition

For unsigned operands A and B, the module implements the following mathematical properties:

- $\gcd(A, 0) = A$
- $\gcd(0, B) = B$
- $\gcd(0, 0) = 0$
- $\gcd(A, A) = A$
- $\gcd(A, B) = \gcd(A - B, B)$ when $A > B$
- $\gcd(A, B) = \gcd(A, B - A)$ when $B > A$

1.2 Interface Definition

The module is parametrized with `WIDTH`, which defines the width of the data operands and defaults it to 16.

1.2.1 Global Signals

- `clk`: Clock signal.
- `rst_n`: Active-low synchronous reset.

1.2.2 Input Channel

A transaction is captured on the rising edge where `in_valid` and `in_ready` are simultaneously asserted.

- `in_valid`: Asserted by the producer when `a_in` and `b_in` hold valid data.
- `in_ready`: Asserted by the module when ready to accept a new transaction; high only in the `IDLE` state.
- `a_in`, `b_in`: `WIDTH`-bit operands.

1.2.3 Output Channel

A transaction completes on the rising edge where `out_valid` and `out_ready` are simultaneously high.

- `out_valid`: Asserted by the module when `gcd_out` holds a valid result; high during the `DONE` state.
- `out_ready`: Asserted by the consumer when it is ready to accept the result.
- `gcd_out`: `WIDTH`-bit GCD result, which remains stable and valid for the entire period `out_valid` is asserted.

1.3 Functional Description and State Machine

The module behaves according to a Finite State Machine (FSM) comprising three states:

- **IDLE**: The module waits for input and asserts `in_ready = 1`.
- **RUN**: The module performs iterative subtraction. In each cycle, if `a_reg > b_reg`, it computes `a_next = a_reg - b_reg`. If `b_reg > a_reg`, it computes `b_next = b_reg - a_reg`.
- **DONE**: The result is ready, and the module asserts `out_valid = 1`.

1.3.1 Transitions

- **IDLE** → **DONE**: Occurs in exactly 1 cycle upon an input handshake if the result is known immediately (`a_in == 0`, `b_in == 0`, or `a_in == b_in`).
- **IDLE** → **RUN**: Occurs upon an input handshake if operands are non-zero and unequal.
- **RUN** → **DONE**: Fires when the next-cycle values of the working registers become equal (`a_next == b_next`).
- **DONE** → **IDLE**: Triggered by the output handshake (`out_valid == 1` and `out_ready == 1`).

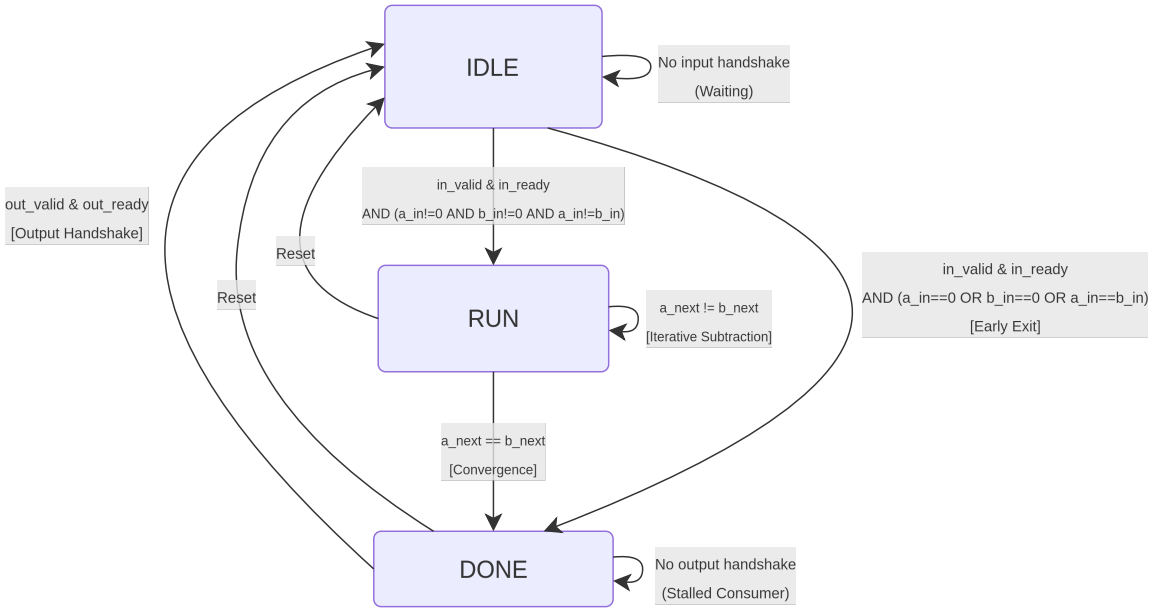

Figure 1: Finite State Machine diagram of the GCD Module

1.4 Latency and Throughput

The module processes one transaction at a time. Latency is defined as the number of cycles from the input handshake edge to the first cycle where `out_valid` is asserted.

- **Early Exit**: 1 cycle (for operands yielding an immediate result).
- **General Case**: $1 + N$ cycles, where $N \geq 1$ represents the number of subtraction steps required for convergence.

1.5 Signals table

Name	Direction	Width	Description
`clk`	Input	1	Clock signal
`rst_n`	Input	1	Active-low sync reset
`in_valid`	Input	1	Asserted by the producer when a_in and b_in hold valid data
`in_ready`	Output	1	Asserted by the module when it can accept a new transaction. High only in the IDLE state.
`a_in`	Input	WIDTH	First operand
`b_in`	Input	WIDTH	Second operand
`out_valid`	Output	1	Asserted by the module when gcd_out holds a valid result. High for the entire DONE state.
`out_ready`	Input	1	Asserted by the consumer when it is ready to accept the result.
`gcd_out`	Output	WIDTH	GCD result. Valid and stable for the entire period that out_valid is asserted.

Table 1: GCD Module Input and Output Signals

2 Traceability Matrix

Req ID	Requirement	Feature	Test ID
REQ-001	GCD is performed correctly for operands A and B	GCD Functioning (General case)	`gcd_random_test`
REQ-002	Edge cases GCD(0,0), GCD(A, 0) (with $A \neq 0$) and GCD(0, B) (with $B \neq 0$) are covered. Output is correct	GCD Functioning (Edge cases)	`gcd_directed_test`
REQ-003	Edge case GCD(A, A) is covered. Output is correct	GCD Functioning (Edge cases)	`gcd_directed_test`
REQ-004	Edge case GCD(A, B), where $A == 2^{WIDTH} - 1$ or $B == 2^{WIDTH} - 1$	GCD Functioning (Edge cases)	`gcd_directed_test`
REQ-005	GCD(A,B) is performed correctly for $A > B$ and $B > A$	GCD Functioning (General case)	`gcd_random_test`
REQ-006	When `rst_n` is low, shift to the IDLE state	Reset behaviour	SVA: `assert_reset_behavior`
REQ-007	When `rst_n` is low, clear internal registers to 0	Reset behaviour	SVA: `assert_reset_behavior`
REQ-008	When `rst_n` is low, sets `in_ready = 1`, `out_valid = 0`, and `gcd_out = 0`.	Reset behaviour	SVA: `assert_reset_behavior`

Continued on next page

Table 2: Traceability Matrix: Features and Tests (Continued)

Req ID	Requirement	Feature	Test ID
REQ-009	Transactions captured only when `in_ready` && `in_valid` are high	Handshake Protocol	SVA: `assert_in_valid_no_drop` and `assert_out_valid_no_drop`
REQ-010	State transitions from IDLE to DONE takes only 1 cycle when $A == 0$ or $B == 0$ or $A == B$	1 cycle GCD (Latency)	`gcd_directed_test`
REQ-011	General case of $N \geq 1$ subtraction steps occur in $1 + N$ cycles	$1 + N$ Cycles (Latency)	`gcd_random_test`
REQ-012	`gcd_out` remains stable until `out_ready` is asserted	Output Stability	SVA: `assert_output_stable`
REQ-013	DUT accepts input on handshake and immediately drops `in_ready` to indicate processing has started	FSM Transitions	SVA: `assert_in_ready_drop`
REQ-014	DUT asserts `out_valid` within the maximum theoretical latency limit (2^{WIDTH} cycles) without hanging	FSM Transitions	Procedural watchdog: `MAX_TIMEOUT`
REQ-015	DUT drops `out_valid` on the clock cycle immediately following the `out_ready` handshake.	FSM Transitions	SVA: `assert_out_valid_drop`
REQ-016	Module is ready to accept input (`in_ready == 1`) immediately on the first rising edge after `rst_n` is released, and it is in IDLE state	Post-Reset	SVA: `assert_reset_behavior`
REQ-017	New input handshake can occur on the earliest cycle immediately after an output handshake	Throughput	`gcd_random_test`
REQ-018	On `in_valid == 1`, `a_in`, and `b_in` must remain stable while stalled (`in_ready == 0`)	Protocol	SVA: `assert_input_stable`

Table 2: Traceability Matrix: Features and Tests

Req ID	Test ID	Coverage Goal
REQ-001	`gcd_random_test`	100% Functional Coverage
REQ-002	`gcd_directed_test`	100% Functional Coverage
REQ-003	`gcd_directed_test`	100% Functional Coverage
REQ-004	`gcd_directed_test`	100% Functional Coverage
REQ-005	`gcd_random_test`	100% Functional Coverage
REQ-006	SVA: `assert_reset_behavior`	100% Code Coverage and Assertion Pass

Continued on next page

Table 3: Traceability Matrix: Coverage Goals (Continued)

Req ID	Test ID	Coverage Goal
REQ-007	SVA: `assert_reset_behavior`	100% Code Coverage and Assertion Pass
REQ-008	SVA: `assert_reset_behavior`	100% Code Coverage and Assertion Pass
REQ-009	SVA: `assert_in_valid_no_drop` and `assert_out_valid_no_drop`	100% Assertion Pass
REQ-010	`gcd_directed_test`	100% Scoreboard Match
REQ-011	`gcd_random_test`	100% Scoreboard Match
REQ-012	SVA: `assert_output_stable`	100% Assertion Pass
REQ-013	SVA: `assert_in_ready_drop`	100% Assertion Pass
REQ-014	Procedural watchdog: `MAX_TIMEOUT`	Zero UVM Errors (No timeout)
REQ-015	SVA: `assert_out_valid_drop`	100% Assertion Pass
REQ-016	SVA: `assert_reset_behavior`	100% Assertion Pass
REQ-017	`gcd_random_test`	100% Scoreboard Match & Zero UVM Errors
REQ-018	SVA: `assert_input_stable`	100% Assertion Pass

Table 3: Traceability Matrix: Coverage Goals

Note: SVA stands for System Verilog Assertions.

2.1 Verification Strategy

The Verification Strategy will be divided on different approaches, each one will be described in the following sections.

2.1.1 Constrained Random Testing

To tackle the GCD Functioning, a Constrained Random Testing Verification will be implemented. The testbench will create randomized input operands for A and B and a UVM scoreboard, containing a Golden Reference Model that will compute the expected GCD result, will be used to make a comparison with the DUT output. In fact, the scoreboard will collect the randomized inputs, compute the expected GCD result of the two operands and compare it with the DUT output.

2.1.2 Directed Testing

To cover GCD functioning corner cases, the early exit scenario (1 cycle GCD Latency) and reset behaviours, I will implement a Directed Testing Verification setup, to correctly cover these particular scenarios.

For the GCD part, I will create specific values for the operands A and B to cover the following cases:

- GCD(0, 0)
- GCD(A, 0) (with $A \neq 0$)
- GCD(0, B) (with $B \neq 0$)
- GCD(A, A)
- GCD(A, B) where $A == 2^{WIDTH} - 1$ or $B == 2^{WIDTH} - 1$

Using the same scoreboard setup, I will create specific tests to cover the early exit scenario, where the GCD is expected to be computed in 1 cycle.

Lastly, I will create a specific test to cover the reset behaviour, where the DUT will be reset and the output will be checked to verify that it is in the expected state.

To check all of these tests, I will embed in the UVM scoreboard previously mentioned to compare the DUT output with the expected output.

2.1.3 Assertion Based Verification

Assertions are ideal for checking handshake signals and protocols compliance. In fact, they evaluate over multiple simulation cycles. That is why I decided to implement an Assertion Based Verification strategy using System Verilog Assertions to cover temporal properties, control logic and compliance of the protocols (such as correctness of handshakes). The following properties are tested with this approach:

- handshake protocol for FSM transitions and data transfer (both input and output channels)
- End to end handshake protocol compliance and timeout limits (verifying implicitly from the outside that the internal FSM does not hang or enter deadlocks)
- data stability during stalled input handshake (`a_in` and `b_in` remain stable when `in_valid == 1` and `in_ready == 0`)
- output stability and correct behaviour during reset

These properties will be evaluated during the run of the UVM simulation. However, they are designed to be formal ready, meaning that can be used in a formal verification tool to prove the state space of the control logic without any change.

2.1.4 Summary table

To summarize all of these verification strategies, the following table shows the mapping between the requirements and the verification strategy used to verify them.

Req IDs	**Verification Strategy**	**Target Features**
REQ-001, REQ-005, REQ-011, REQ-017	Constrained Random Testing	GCD Functioning (General Case), 1+N Cycles (Latency), Throughput
REQ-002, REQ-003, REQ-004, REQ-006, REQ-007, REQ-008, REQ-010	Directed Testing	GCD Functioning (Edge Cases), 1 Cycle GCD (Latency), Reset Behaviour
REQ-009, REQ-012, REQ-013, REQ-014, REQ-015, REQ-016, REQ-018	Assertion Based Verification	Handshake Protocol, Protocol, FSM Transitions, Output Stability, Post-Reset,

Table 4: Verification Strategy Summary

2.2 Closure Criteria

To consider the verification process complete and successful, the following quantitative metrics must be achieved:

- **100% Test Pass Rate:** All UVM tests (smoke, directed, and constrained-random) must pass successfully without any fatal errors or timeouts.

- **100% Functional Coverage:** All defined covergroups and coverpoints (including mathematical edge cases, mathematical states, and operand categories) must reach 100% coverage.
- **High Code Coverage:** Achieve at least 90% Line, Branch, and Toggle coverage. Any uncovered code must be manually reviewed and justified (e.g., unreachable code).
- **100% Assertion Pass Rate:** Zero assertion failures during all test runs, confirming protocol and timing compliance.

3 UVM Testbench Implementation

3.1 UVM Architecture

For the GCD module, I implemented a UVM testbench architecture. One methodology could be of implement two different agents, one for input channel and one for the output one, as they have independent handshake protocols.

However, I decided to implement a single agent, because both channels share the same clock and reset signals, and the dimension of the module itself is relatively compact.

The following diagram shows the UVM architecture that I implemented.

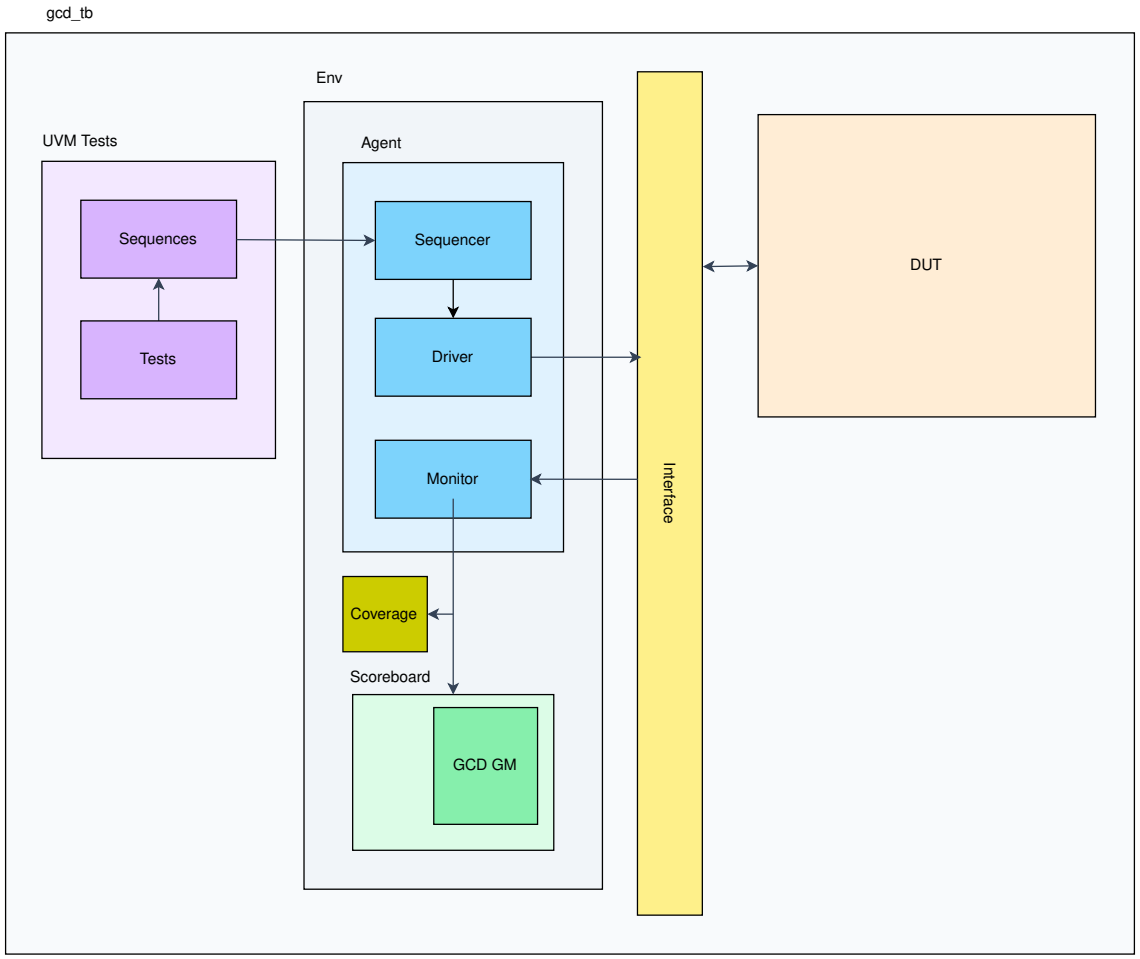

Figure 2: UVM testbench architecture diagram

3.2 Architecture Overview

The primary components are structured as follows:

- **UVM Tests and Sequences:** The test layer defines the scenario (random, directed, smoke) and starts the corresponding sequences. The sequences generate randomized `gcd_sequence_item` transactions and pass them to the environment.
- **Environment (Env):** contains the Agent and the Scoreboard.
  - **Agent:** manages the active stimulus and passive observation of the bus. It contains the Sequencer, Driver, and Monitor.
    - **Sequencer:** takes generated sequence items from the test sequences and queues them for the driver.
    - **Driver:** pulls sequence items from the sequencer and toggles the interface pins of the DUT according to the valid/ready handshake protocols.
    - **Monitor:** passively observes the interface pins, reconstructing completed input and output handshakes back into transaction objects to broadcast to the scoreboard.
  - **Scoreboard:** contains the Reference Model and Covergroups to evaluate correctness of the DUT.
- **Interface:** External to the UVM environment, this static module connects the UVM testbench to the DUT pins and houses the SystemVerilog Assertions (SVA) and Watchdog timers.

3.3 Stimulus Generation (Sequences and Sequence Items)

The `gcd_sequence_item` forms the basic transaction payload. It contains the `WIDTH`-bit operands (`a` and `b`) and a randomizable integer, `out_ready_delay`. This delay variable is constrained to simulate a stalled consumer, testing the `DONE` state stability of the DUT.

```
class gcd_sequence_item extends uvm_sequence_item;
    `uvm_object_utils(gcd_sequence_item)

    rand bit [15:0] a;
    rand bit [15:0] b;
    rand int out_ready_delay;
    bit [15:0] gcd_out;

    // Constraint: mostly ready immediately, sometimes stalled
    constraint c_ready_delay {
        out_ready_delay dist {0 := 50, [2:5] := 50};
    }
```

I've implemented three primary sequences:

- **Smoke Sequence:** A hardcoded, simple transaction to verify basic connectivity and that the DUT can complete a handshake and output a valid result.
- **Directed Edge Case Sequence:** Specifically targets mathematical bounds. It injects combinations of zeros, equal operands, and maximum boundary values ($2^{WIDTH} - 1$) to ensure the early exit (1-cycle latency) and edge cases operate without any protocol violations.
- **Constrained Random Sequence:** Generates between 250 and 500 randomized transactions to thoroughly stress the general $1 + N$ subtraction algorithm.

3.4 Agent Components

3.4.1 Driver

The `gcd_driver` retrieves sequence items from the sequencer and drives the corresponding values onto the physical pins of the interface. A critical feature of the driver is its handling of the dual handshakes. It asserts `in_valid` and holds the input operands stable until the DUT raises `in_ready`. Once the input handshake ends, the driver implements the `out_ready_delay` requested by the sequence item before asserting `out_ready`, testing the ability of the DUT to hold `gcd_out` stable.

3.4.2 Monitor

The `gcd_monitor` passively observes the bus via a dedicated monitor clocking block. It waits for simultaneous `in_valid` and `in_ready` assertions to capture the incoming operands, and then waits for the `out_valid` and `out_ready` assertions to capture the result. The complete transaction is then written to an Analysis Port for the scoreboard to evaluate.

The following code shows the passive handshake observation of the monitor using `do...while` loops:

```
// Wait for input handshake
do begin
    @(vif.mon_cb);
end while (!(vif.mon_cb.in_valid === 1'b1 && vif.mon_cb.in_ready === 1'b1));

tr.a = vif.mon_cb.a_in;
tr.b = vif.mon_cb.b_in;

// Wait for output handshake
do begin
    @(vif.mon_cb);
end while (!(vif.mon_cb.out_valid === 1'b1 && vif.mon_cb.out_ready === 1'b1));

tr.gcd_out = vif.mon_cb.gcd_out;
analysis_port.write(tr);
```

3.5 Scoreboard and Coverage

The `gcd_scoreboard` acts as both the evaluator and the coverage collector. It receives transactions from the monitor and passes the input operands to a custom Golden Reference Model.

The Reference Model calculates the expected Greatest Common Divisor using the Modulo operator, instead of the subtractive approach of the DUT. The expected result is then compared against the `gcd_out` of the DUT.

```
function int calculate_gcd(int a, int b);
    if (a == 0) return b;
    if (b == 0) return a;
    while (b != 0) begin
        int temp = b;
        b = a % b; // remainder
        a = temp;  // swap
    end
    return a;
endfunction
```

At the same time, the scoreboard samples the `gcd_cg` Covergroup. This covergroup ensures thorough functional coverage by tracking:

- Occurrences of zero, max value, and intermediate values for both `a` and `b`.
- The cross-coverage between the two operands.
- Mathematical states, specifically whether $A > B$, $B > A$, or $A = B$.

```
covergroup gcd_cg;
    option.per_instance = 1;

    cp_a: coverpoint cov_transaction.a {
        bins zero = {0};
        bins max  = {((1<<WIDTH)-1)}; // 2^(WIDTH)-1
        bins others = {[1 : ((1<<WIDTH)-2)]};
    }

    cp_b: coverpoint cov_transaction.b {
        bins zero = {0};
        bins max  = {((1<<WIDTH)-1)}; // 2^(WIDTH)-1
        bins others = {[1 : ((1<<WIDTH)-2)]};
    }

    cross_a_b: cross cp_a, cp_b;

    cp_mathematical_states: coverpoint (cov_transaction.a > cov_transaction.b) {
        bins a_greater_b = {1};
        bins b_greater_a = {0};
    }

    cp_equal: coverpoint (cov_transaction.a == cov_transaction.b) {
        bins a_equals_b = {1}; // GCD(A,A)
    }
endgroup
```

3.6 Interface, Assertions, and Watchdogs

While the UVM environment handles the dynamic stimulus and checking, the static `gcd_if` interface is heavily utilized to monitor protocol compliance and prevent deadlocks. This is reached through a combination of SystemVerilog Assertions (SVAs) and a dynamic procedural watchdog timer.

3.6.1 SystemVerilog Assertions (SVAs)

Concurrent assertions have been developed inside the uvm interface to directy monitor the `ready/valid` handshake protocols and ensure data stability during stalls. For instance, to satisfy the requirement that `gcd_out` remains stable until the consumer is ready, the following property is asserted:

```
// REQ 12 output stability
property p_output_stable;
    @(posedge clk) disable iff (!rst_n || $isunknown(out_valid) ||
    ↪  $isunknown(out_ready) || $isunknown(gcd_out))
    (out_valid && !out_ready) |=> (out_valid && $stable(gcd_out));
endproperty

assert_output_stable: assert property(p_output_stable)
    else $error("PROTOCOL VIOLATION: gcd_out changed while stalled!");
```

To verify correct reset behavior, the following property ensures that upon a reset, the module immediately transitions to the IDLE state by raising `in_ready` and clearing all outputs:

```
// REQ 6,7,8,16 reset behavior
property p_reset_behavior;
    @(posedge clk) !rst_n |=> (in_ready == 1'b1 && out_valid == 1'b0 && gcd_out ==
    ↪  '0);
endproperty

assert_reset_behavior: assert property(p_reset_behavior)
    else $error("RESET VIOLATION: Signals did not default to IDLE state!");
```

To guarantee the correctness of the standard handshake, this property states that once a `valid` signal is asserted (on both the input and output channel), it cannot be de-asserted before the corresponding `ready` signal acknowledges the transfer:

```
// REQ 9 handshake protocol
property p_valid_no_drop(valid, ready);
    @(posedge clk) disable iff (!rst_n)
    (valid && !ready) |=> valid;
endproperty

assert_in_valid_no_drop: assert property(p_valid_no_drop(in_valid, in_ready))
    else $error("PROTOCOL VIOLATION: in_valid dropped before in_ready!");
assert_out_valid_no_drop: assert property(p_valid_no_drop(out_valid, out_ready))
    else $error("PROTOCOL VIOLATION: out_valid dropped before out_ready!");
```

To confirm the DUT correctly leaves the IDLE state, this assertion verifies that `in_ready` drops immediately following a successful input handshake:

```
// REQ 13 FSM start
property p_in_ready_drop;
    @(posedge clk) disable iff (!rst_n)
    (in_valid && in_ready) |=> (!in_ready);
endproperty

assert_in_ready_drop: assert property(p_in_ready_drop)
    else $error("FSM VIOLATION: in_ready did not drop after input handshake!");
```

In a similar way, to prevent duplicate reads, this property makes sure that the module drops the `out_valid` signal in the cycle immediately following a successful output handshake:

```
// REQ 15 FSM to idle
property p_out_valid_drop;
    @(posedge clk) disable iff (!rst_n)
    (out_valid && out_ready) |=> (!out_valid);
endproperty

assert_out_valid_drop: assert property(p_out_valid_drop)
    else $error("FSM VIOLATION: out_valid did not drop after output handshake!");
```

To evaluate the protocol compliance from the producer, the next assertion monitors the input channel to strictly enforce that the operands remain perfectly stable during a stalled handshake:

```
// REQ 18 input stability
property p_input_stable_when_stalled;
    @(posedge clk) disable iff (!rst_n || $isunknown(in_ready) || $isunknown(in_valid)
    ↪  || $isunknown(a_in) || $isunknown(b_in))
    (in_valid && !in_ready) |=> (in_valid && $stable(a_in) && $stable(b_in));
endproperty

assert_input_stable: assert property(p_input_stable_when_stalled)
    else $error("PROTOCOL VIOLATION: in_valid dropped or input operands changed while
    ↪  stalled!");
```

Finally, the following property can be considered as a concurrent watchdog to formally verify that the DUT always produces a valid output within the theoretical maximum latency limit:

```
property p_no_infinite_hang;
    @(posedge clk) disable iff (!rst_n || $isunknown(in_valid) ||
    ↪  $isunknown(out_ready))
    (in_valid && in_ready) |=> ##[1:MAX_TIMEOUT] out_valid;
endproperty

assert_no_infinite_hang: assert property(p_no_infinite_hang)
    else $error("TIMEOUT: Module exceeded maximum cycles without asserting
    ↪  out_valid!");
```

3.6.2 Dynamic Procedural Watchdog

To be sure that the DUT does not hang indefinitely (during complex mathematical edge cases, important when WIDTH as a high value), I implemented a watchdog timer. At first I've used a hardcoded value, but then I developed a watchdog that dynamically calculates the maximum theoretical cycles required for convergence based on the specific inputs captured during the handshake.

If the DUT fails to assert `out_valid` within this calculated window (plus a small margin of 10 cycles for handshake synchronization), the simulation throws a fatal error (`$fatal`).

```
int count = 0;
int unsigned local_timeout = count_cyc(mon_cb.a_in, mon_cb.b_in) + 10;

$display("[%0t] TIMEOUT LIMIT = %0d cycles", $time, local_timeout);

while (count <= local_timeout) begin
    @(mon_cb);
    count++;
    if (mon_cb.out_valid || !rst_n)
        break;
end

if (count > local_timeout) begin
    $fatal(1, "[TIMEOUT] DUT hung! Exceeded max theoretical cycles (%0d)",
    ↪  local_timeout);
end
```

```
function automatic int unsigned count_cyc(bit [WIDTH-1:0] a, bit [WIDTH-1:0] b);
    int unsigned counter = 0;
    if (a == 0 || b == 0 || a == b) return 1;
    while (a != 0 && b != 0 && a != b) begin
        if (a > b) a -= b; else b -= a;
        counter++;
    end
    return counter;
endfunction
```

4 UVM Tests and Sequences

The verification environment uses three different UVM Tests to verify the DUT. Each test is a wrapper that instantiates the UVM environment, raises an objection to prevent the simulation from ending, starts a specific sequence on the sequencer of the agent, and then drops the objection once the sequence finishes. The command line examples to run each of the tests are provided in the next section (Functional Coverage Report).

4.1 Smoke (or bring-up) Test (`gcd_smoke_test`)

The Smoke Test is designed to verify the fundamental connectivity of the testbench and ensure the DUT can successfully complete a basic transaction without hanging.

The Smoke Test executes the `gcd_smoke_seq`, which overrides the default sequence item constraints to inject a single, hardcoded transaction. The operands are hardcoded to $A = 15$ and $B = 5$, and the `out_ready_delay` is explicitly forced to 0. This represents the simplest possible handshake scenario, to confirm that the interface protocols, reset behaviors, and FSM transitions function correctly before introducing randomness or complex timing.

4.2 Directed Edge Case Test (`gcd_directed_test`)

To ensure the DUT properly handles boundary conditions and early exit scenarios (with only 1 cycle latency), the `gcd_directed_test` executes the `gcd_directed_edge_case_seq`.

Instead of relying on random probability to hit specific mathematical extremes, this sequence explicitly utilizes a custom `send_directed_item()` task to add targeted operand combinations. The following edge cases are verified:

- **Zero Operands:** $\gcd(0, 0)$, $\gcd(A, 0)$, and $\gcd(0, B)$. These test the immediate early-exit transition from `IDLE` directly to `DONE`.
- **Equal Operands:** $\gcd(A, A)$. This verifies the immediate equality check bypassing the iterative subtraction loop.
- **Maximum Boundaries:** Combinations utilizing $(2^{WIDTH}) - 1$. This stresses the datapath width limits and verifies the combinatorial subtraction logic under maximum bit-toggle conditions.

```
task body();
    `uvm_info("SEQ", "Executing directed edge case sequence", UVM_LOW)

    send_directed_item(0, 0);                  // GCD(0, 0)
    send_directed_item(18, 0);                 // GCD(A != 0, B = 0)
    send_directed_item(0, 24);                 // GCD(A = 0, B != 0)
    send_directed_item(27, 27);                // GCD(A = B)

    // Max values (WIDTH-1)
    send_directed_item((1 << WIDTH) - 1, 5);  // GCD(MAX, B)
    send_directed_item(5, (1 << WIDTH) - 1);  // GCD(A, MAX)
    send_directed_item((1 << WIDTH) - 1, (1 << WIDTH) - 1); // GCD(MAX, MAX)

    send_directed_item(0, (1 << WIDTH) - 1); // GCD(0, MAX)
    send_directed_item((1 << WIDTH) - 1, 0); // GCD(MAX, 0)
endtask
```

4.3 Constrained Random Test (`gcd_random_test`)

The main part of the functional verification is performed by the `gcd_random_test`, which runs the `gcd_random_seq`. This test stresses the general iterative subtractive algorithm $(1 + N$ cycles latency$)$ and the output stalling logic.

```
class gcd_random_seq extends uvm_sequence #(gcd_sequence_item);
    `uvm_object_utils(gcd_random_seq)

    rand int number_transactions;

    constraint c_number_of_transactions {
        number_transactions inside {[250:500]}; // rand between 250 and 500
    }

    function new(string name = "gcd_random_seq");
        super.new(name);
    endfunction

    task body();
        if (!randomize()) begin
            `uvm_error("SEQ", "Randomization failed for number of transactions")
        end

        `uvm_info("SEQ", $sformatf("Executing Random Sequence with %0d transactions",
        ↪  number_transactions), UVM_LOW)

        for (int i = 0; i < number_transactions; i++) begin
            req = gcd_sequence_item::type_id::create("req");
            $display("Randomizing transaction %0d/%0d", i+1, number_transactions);
            start_item(req);
            if (!req.randomize()) begin
                `uvm_error("SEQ", "Randomization failed for transaction")
            end
            $display("Finishing transactions");
            finish_item(req);
            $display("Transactions finished");
        end
    endtask
endclass
```

The sequence randomizes the `number_transactions` variable to execute between 250 and 500 transactions. During this sequence, the `out_ready_delay` parameter within the sequence items frequently forces the driver to stall the output handshake, in order to test the `DONE` state stability requirement and the concurrent assertions monitoring it.

5 Functional Coverage Report

The following table summarizes the overall coverage extracted from the QuestaSim UCDB reports across the testbench and DUT instances.

Coverage Metric	Enabled Bins	Hits	Misses	Coverage (%)
Covergroups	18	17	1	94.44%
Assertions	8	6	2	75.00%
Branches	22	20	2	90.90%
Statements	25	24	1	96.00%
Toggles	271	253	18	93.35%
FSM States	3	3	0	100.00%
FSM Transitions	5	5	0	100.00%

Table 5: Coverage Summary by Instance

These results have been obtained with a Makefile that runs all the tests and merges the UCDB files into a single coverage database. The relevant part of the Makefile are shown below:

```
setup:
        vlib work
        vmap work ./work
        vlog $(VFLAGS) $(UNIT).sv
        vlog +acc $(TESTBENCH)_$(UNIT).sv

run_all: setup
        $(MAKE) run TEST=gcd_smoke_test UCDB_OUT=smoke.ucdb
        $(MAKE) run TEST=gcd_directed_test UCDB_OUT=directed.ucdb
        $(MAKE) run TEST=gcd_random_test UCDB_OUT=random.ucdb
        vcover merge gcd_cov.ucdb bringup.ucdb directed.ucdb random.ucdb

report:
        vcover report -details -output gcd_cov_report.txt gcd_cov.ucdb
        vcover report gcd_cov.ucdb >> gcd_cov_report.txt
```

In any case, these are the final coverage percentages:

```
TOTAL COVERGROUP COVERAGE: 97.77%  COVERGROUP TYPES: 1

TOTAL ASSERTION COVERAGE: 75.00%  ASSERTIONS: 8

Total Coverage By Instance (filtered view): 91.60%
```

5.1 Covergroup Coverage Analysis

The functional coverage model (`gcd_cg`) defined 18 bins to track operand edge cases, mathematical states ($A > B$, $A = B$), and cross-coverage between operands. We can observe that the environment hit 17 of 18 bins (94.44%).

The single missed bin was the cross-coverage scenario between `a = others` and `b = max`. Looking at the extended coverage report, it confirms this miss, displaying the "ZERO" keyword for this specific bin, indicating it was never sampled by the scoreboard.

However, as showed before, the stimulus for this exact scenario (`send_directed_item(5, (1 << WIDTH) - 1)`) is explicitly generated and driven into the DUT.

Because this specific transaction is injected near the very end of the directed sequence, I hypothesize that the absence of this hit to be a simulation drain-time synchronization issue. The testbench likely dropped the UVM objection and terminated the simulation before the DUT could finish computing the result, preventing the monitor from capturing the final output handshake and passing the transaction to the scoreboard for sampling.

5.2 Assertion Coverage Analysis

Assertion coverage reached 75.00%, with 6 out of 8 protocol monitors (properties) passing successfully throughout the test suites. The two missed assertions were `assert_in_valid_no_drop` and `assert_input_stable`.

Both of these SVA evaluate producer compliance when the DUT stalls the input channel (`in_valid == 1` while `in_ready == 0`). However, the UVM driver is designed to be compliant and sequential: it waits for the DUT to explicitly return to the `IDLE` state (`in_ready == 1`) before driving the next valid sequence item. Because the driver never attempts to force a transaction while the DUT is processing, the stall precondition is impossible to reach in the current architecture. To trigger these assertions, an error-injection sequence would be required.

5.3 Branch and Structural Coverage Analysis

Branch coverage reached 90.90%. Analysis of the uncovered branches reveals that they are within the subtractive loop of the `RUN` state. The RTL evaluates `if (a_reg > b_reg)` and `else if (b_reg > a_reg)`. The missing branch is the implicit `else` condition (where `a_reg == b_reg`).

This miss is a result of the RTL design rather than a verification hole.

This FSM is coded to transition immediately to `DONE` on the cycle where the next state values are equal (`a_next == b_next`). Therefore, the values are never registered as equal, while remaining inside the `RUN` state, making the implicit `else` branch structurally unreachable.

5.4 FSM Transition Verification

All 5 defined state transitions (100%) were successfully covered. Standard operational transitions (`IDLE` $\rightarrow$ `RUN`, `RUN` $\rightarrow$ `DONE`, and `DONE` $\rightarrow$ `IDLE`) were heavily exercised by the constrained random sequences. The early-exit transition (`IDLE` $\rightarrow$ `DONE`) was explicitly hit by the directed edge-case tests.

The most difficult transition to verify was the recovery during calculation (`RUN` $\rightarrow$ `IDLE`). To achieve this, I injected a hardcoded procedural block into the static top module. By using a `fork...join_any` structure, the testbench monitors the `in_ready` signal, waits for it to drop (indicating the DUT has entered the `RUN` state), delays for a few cycles, and then manually asserts `rst_n = 0`. This successfully proved the ability of the DUT to safely abort an active calculation and reset its datapath.

6 Verification Failure - Bug Injection

To demonstrate the robustness of the testbench, specifically the procedural watchdog and concurrent assertions, I forced a bug into the DUT to simulate a designer error.

6.1 Bug Description: Premature IDLE Transition

In the correct implementation, when the iterative subtraction loop inside the `RUN` state achieves convergence (`a_next == b_next`), the FSM must register the result and transition to the `DONE` state to initiate the output handshake.

For this failure scenario, the RTL was modified to simulate a designer error where the FSM bypasses the `DONE` state and transitions directly back to `IDLE`. Consequently, the `out_valid` signal is never asserted for the consumer.

The injected bug in `gcd.sv` is shown below:

```
// ------------------------------------------------------------
// RUN: subtractive Euclidean algorithm
// ------------------------------------------------------------
RUN: begin
    a_reg <= a_next;
    b_reg <= b_next;

    // Check convergence
    if (a_next == b_next) begin
        result_reg <= a_next;
        // Forced BUG: Should be state <= DONE;
        state      <= IDLE;
    end
end
```

6.2 Testbench Detection and Logs

Because the DUT skips the `DONE` state, the output handshake logic of the uvm driver becomes permanently stalled waiting for `out_valid == 1`. This would cause an infinite loop.

However, the injected bug is immediately caught by two distinct mechanisms in the `gcd_if` interface:

- the procedural watchdog calculates the exact maximum cycles required for the specific operands. When the DUT fails to assert `out_valid` within this window, it throws a `$fatal` error.
- the SVA `assert_no_infinite_hang` independently monitors the theoretical global timeout limit and throws an error when the transaction exceeds the maximum theoretical latency.

Running the `gcd_random_test` with this bug prints the following transcript output:

```
# UVM_INFO @ 0: reporter [RNTST] Running test gcd_random_test...
# [115] DRIVING: a=24 b=18 in_valid=1
# [125] WATCHDOG ARMED a=24 b=18
# [125] TIMEOUT LIMIT = 13 cycles
# [125] INPUT HANDSHAKE COMPLETE
# Waiting for output handshake... out_valid: 0, out_ready: 1
#
# ** Fatal: [TIMEOUT] DUT hung! Exceeded max theoretical cycles (13)
#    Time: 255 ns  Scope: testbench_gcd.gcd_if_inst File: testbench_gcd.sv Line: 76
#
# ** Error: TIMEOUT: Module exceeded maximum cycles without asserting out_valid!
#    Time: 255 ns  Scope: testbench_gcd.gcd_if_inst.assert_no_infinite_hang File:
↪  testbench_gcd.sv Line: 155
#
# ** Note: $fatal : Simulation stopped.
```

6.3 Bug Report Ticket

If this were discovered during a real verification cycle, it would be logged to the design team as follows, to request a fix:

[BUG-001] Premature IDLE Transition (Skipping DONE State)

Component:	GCD Module / FSM Logic
Severity:	**CRITICAL** (Complete datapath deadlock)
Description:	The FSM fails to enter the `DONE` state after mathematical convergence in the `RUN` state. It illegally jumps to `IDLE`.
Steps to Reproduce:	Run any transaction where $A \neq B$.
Expected Behavior:	When `a_next == b_next`, `state` should be assigned to `DONE`, triggering `out_valid = 1`.
Actual Behavior:	`state` is assigned to `IDLE`. The output handshake is never initiated, stalling downstream consumers indefinitely.

7 Appendix

To analyze the waveforms, I have used the Surfer software.
QuestaSim has been used to run the simulations and generate the coverage reports.

7.1 Number of hours required

For the completion of this laboratory, it has been needed 40 hours.

7.2 Use of AI

Gemini AI has been used to fix bugs in the code, improve formulation of some sentences in the report and help in cover the RUN $\rightarrow$ IDLE transition in the FSM.